



Chapter 10:

Object-Oriented Thinking

Part II

10.7. Processing Primitive DataType Values as Object

👉 Primitive Data Types and Wrapper Classes

- ✨ The **primitive data type value** is not an object, but it can be converted to an object using a wrapper class.
- ✨ The **Wrapper class** is a class that encapsulate primitive data types, allowing them to be treated as objects. Each primitive type has a corresponding wrapper class (e.g., `Integer` for `int`, `Double` for `double`, `Character` for `char`).
- ✨ Wrapper classes are part of the Java standard library and provide methods for converting between primitive types and objects, as well as useful constants and methods for manipulation.

👉 **Numeric Wrapper Classes**

✨ **Numeric Wrapper Classes** are used to represent primitive numeric types as objects. They provide methods for conversion, parsing, and mathematical operations.

✨ Common numeric wrapper classes:

- ✨ `Byte` : Represents a `byte` value as an object.
- ✨ `Short` : Represents a `short` value as an object.
- ✨ `Integer` : Represents an `int` value as an object.
- ✨ `Long` : Represents a `long` value as an object.
- ✨ `Float` : Represents a `float` value as an object.
- ✨ `Double` : Represents a `double` value as an object.

👉 Feature of Wrapper Class

- ✨ **Constants:** Each wrapper class provides constants for the minimum and maximum values of the corresponding primitive type (e.g., `Integer.MIN_VALUE` , `Integer.MAX_VALUE`).
- ✨ **Conversion Methods:** Wrapper classes provide methods to convert between primitive types and objects (e.g., `intValue()` , `doubleValue()` , `toString()`).
- ✨ **Parsing Methods:** Methods like `parseInt()` and `parseDouble()` allow conversion from strings to primitive types, enabling easy parsing of user input or data from external sources.
- ✨ **Comparison Methods:** The `compareTo` method allows comparison between two wrapper objects, returning `1` , `0` , or `-1` based on their relative values.

👉 Immutable Wrapper Object

✨ **Wrapper objects** are immutable, meaning their state cannot be changed after creation.

This ensures that once a wrapper object is created, its value remains constant throughout its lifetime.

Example: Immutability of Integer Wrapper Class

```
Integer x = Integer.valueOf(5);  
System.out.println(x); // Output: 5  
x = Integer.valueOf(10); // x now references a new Integer object  
System.out.println(x); // Output: 10
```

Explain:

✨ In this example, the `Integer` object `x` is created with a value of `5`. When we assign a new value of `10` to `x`, it references a new `Integer` object, demonstrating immutability. The original object remains unchanged.

👉 Built-In Integer and Double Wrapper Class

java.lang.Integer	java.lang.Double
-value: int <u>+MAX_VALUE: int</u> <u>+MIN_VALUE: int</u>	-value: double <u>+MAX_VALUE: double</u> <u>+MIN_VALUE: double</u>
+Integer(value: int) +Integer(s: String) +byteValue(): byte +shortValue(): short +intValue(): int +longValue(): long +floatValue(): float +doubleValue(): double +compareTo(o: Integer): int +toString(): String <u>+valueOf(s: String): Integer</u> <u>+valueOf(s: String, radix: int): Integer</u> <u>+parseInt(s: String): int</u> <u>+parseInt(s: String, radix: int): int</u>	+Double(value: double) +Double(s: String) +byteValue(): byte +shortValue(): short +intValue(): int +longValue(): long +floatValue(): float +doubleValue(): double +compareTo(o: Double): int +toString(): String <u>+valueOf(s: String): Double</u> <u>+valueOf(s: String, radix: int): Double</u> <u>+parseDouble(s: String): double</u> <u>+parseDouble(s: String, radix: int): double</u>

10.8. Automatic Conversion between Primitive Types and Wrapper Class Types

👉 Autoboxing and Unboxing

✨ **A primitive-type value** can be automatically converted to an object using a wrapper class, and vice versa, depending on the context.

✨ This automatic conversion is known as **Autoboxing** and **Unboxing**.

✨ **Autoboxing**: The automatic conversion of a primitive type to its corresponding wrapper class object.

✨ **Unboxing**: The automatic conversion of a wrapper class object to its corresponding primitive type.

Example: Autoboxing and Unboxing

```
Integer x = 5; // Autoboxing: int to Integer  
int y = x;     // Unboxing: Integer to int
```

Explain:

- ✨ In this example, the primitive `int` value `5` is automatically converted to an `Integer` object through autoboxing.
- ✨ Later, the `Integer` object `x` is automatically converted back to an `int` value through unboxing.

10.9. The `BigInteger` and `BigDecimal` Classes

👉 Why Use `BigInteger` and `BigDecimal`?

✨ **`BigInteger`** and **`BigDecimal`** classes are used for mathematical operations involving very large integers and high-precision floating-point numbers. Both classes are immutable and part of the `java.math` package.

Usage:

- ✨ In banking, amounts of money are often represented as `BigDecimal` to avoid rounding errors.
- ✨ In scientific calculations, `BigInteger` is used to handle large numbers that exceed the limits of primitive types.

👉 What is `BigInteger` ?

- ✨ **`BigInteger`** is a class that represents integers of arbitrary precision.
- ✨ It is used when the range of primitive integer types (like `int` and `long`) is insufficient for the required calculations.
- ✨ Common methods include `add()` , `subtract()` , `multiply()` , `divide()` , `mod()` , and `compareTo()` .

Example: BigInteger Addition

```
import java.math.BigInteger;
public class BigIntegerExample {
    public static void main(String[] args) {
        BigInteger bigInt1 = new BigInteger("123456789012345678901234567890");
        BigInteger bigInt2 = new BigInteger("987654321098765432109876543210");

        BigInteger sum = bigInt1.add(bigInt2);
        System.out.println("Sum: " + sum);
    }
}
```

Output:

```
Sum: 1111111110111111111011111111100
```

Explain:

- ✨ In this example, two `BigInteger` objects are created with very large integer values.
- ✨ The `add()` method is used to calculate the sum of these two large integers, demonstrating how `BigInteger` can handle values beyond the limits of primitive types.
- ✨ The result is printed, showing the correct sum of the two large integers without any overflow issues.
- ✨ This illustrates the utility of `BigInteger` for high-precision arithmetic operations, especially when dealing with large numbers that exceed the range of standard integer types.

👉 What is `BigDecimal`?

- ✨ **`BigDecimal`** is a class that represents decimal numbers with arbitrary precision.
- ✨ It is used for high-precision calculations, especially in financial applications where rounding errors must be avoided.
- ✨ Common methods include `add()`, `subtract()`, `multiply()`, `divide()`, `setScale()`, and `compareTo()`.

Explain:

- ✨ In this example, two `BigDecimal` objects are created to represent the numbers `1` and `3`.
- ✨ The `divide()` method is used to divide `num1` by `num2`, specifying a scale of `60` and a rounding mode of `ROUND_HALF_UP`.
- ✨ The result is printed, showing the high-precision decimal value of the division operation.
- ✨ This illustrates the utility of `BigDecimal` for performing precise arithmetic operations, especially in scenarios where rounding errors could lead to significant inaccuracies, such as in financial calculations.

10.10. The String Class

👉 What is the Built-in String Class Method in Java?

✨ The `String` class in Java is a built-in class that represents a sequence of characters. It is used to handle text data and provides various methods for string manipulation.

✨ Common methods in the `String` class include:

✨ `length()` : Returns the length of the string.

✨ `charAt(int index)` : Returns the character at the specified index.

✨ `substring(int beginIndex, int endIndex)` : Returns a substring from the string.

✨ `indexOf(String str)` : Returns the index of the first occurrence of a substring.

✨ `toLowerCase()` : Converts the string to lowercase.

✨ `trim()` : Removes leading and trailing whitespace from the string.

Example: Common String Operations

```
String message = "Welcome to Java";  
System.out.println("Length: " + message.length());  
System.out.println("Character at index 4: " + message.charAt(4));  
System.out.println("Substring (0, 7): " + message.substring(0, 7));  
System.out.println("Index of 'Java': " + message.indexOf("Java"));  
System.out.println("Lowercase: " + message.toLowerCase());  
System.out.println("Trimmed: " + message.trim());
```

Output:

```
Length: 16  
Character at index 4: o  
Substring (0, 7): Welcome  
Index of 'Java': 11  
Lowercase: welcome to java  
Trimmed: Welcome to Java
```

👉 Immutable String

✨ Strings object in Java are **immutable**, meaning once a `String` object is created, its contents cannot be changed. Any operation that appears to modify a string actually creates a new `String` object.

Example: Creating a new String with `concat()`

```
String str1 = "Hello";  
String str2 = str1.concat(" World"); // Creates a new String object  
System.out.println(str1); // Output: Hello  
System.out.println(str2); // Output: Hello World
```

Output:

```
Hello  
Hello World
```

Explain:

- ✨ In this example, `str1` is initialized with the value "Hello". When we call `concat()` to append " World", a new `String` object is created and assigned to `str2`.
- ✨ The original string `str1` remains unchanged, demonstrating the immutability of strings in Java.

👉 Interned String

✨ **Interned String Object** refers to strings that are stored in a special memory area called the "**String Pool**" to optimize memory usage.

✨ Interned strings are stored in a pool, so identical string values share the same object in memory.

Example:

```
String str1 = "Hello";  
String str2 = "Hello";  
System.out.println(str1 == str2);
```

Output:

```
true
```

Explain:

- ✨ In this example, both `str1` and `str2` are initialized with the same string literal "Hello". Since string literals are interned by default, both variables refer to the same interned string object in memory.
- ✨ The `==` operator checks for reference equality, and since both variables point to the same interned string, it returns `true`.

Visualizing

String Pool of Interned Strings

Memory Address	String Value
0x4000	"Hello"
0x5000	"World"

String Variables

Variable	Points To
str1	0x4000 ("Hello")
str2	0x4000 ("Hello")
str3	0x5000 ("World")

👉 Replacing String Method

✨ The `replace()` method is used to replace occurrences of a character or substring in a string with another character or substring.

Example: Replacing characters in a string.

```
String s = "Hello World";  
String replaced = s.replace('o', 'a');  
System.out.println(replaced); // Output: Hella World
```

Output:

```
Hella World
```

👉 Splitting Strings

✨ The `split()` method is used to split a string into an array of substrings based on a specified delimiter.

Example: Splitting a string with a delimiter `,`.

```
String s = "Hello,World,Java";  
String[] parts = s.split(",");  
for (String part : parts) {  
    System.out.println(part);  
}
```

Output:

```
Hello  
World  
Java
```

👉 **TIP: Regular Expressions (Regex)**

✨ **Regular expressions** are powerful tools for pattern matching and manipulation in strings.

✨ They allow you to search, replace, and validate strings based on specific patterns.

✨ Java provides the `java.util.regex` package for working with regular expressions.

✨ Common regex operations include:

✨ `matches()` : Checks if the entire string matches a regex pattern.

✨ `find()` : Searches for occurrences of a pattern within a string.

✨ `replaceAll()` : Replaces all occurrences of a pattern with a specified replacement.

👉 Basic Regex Syntax

- ✨ `.` : Matches any character except a newline.
- ✨ `*` : Matches zero or more occurrences of the preceding element.
- ✨ `+` : Matches one or more occurrences of the preceding element.
- ✨ `?` : Matches zero or one occurrence of the preceding element.
- ✨ `[abc]` : Matches any single character in the brackets (a, b, or c).
- ✨ `[^abc]` : Matches any single character not in the brackets (not a, b, or c).
- ✨ `[a-z]` : Matches any single character in the range from a to z.
- ✨ `[0-9]` : Matches any single digit from 0 to 9.
- ✨ `\d` : Matches any digit (equivalent to `[0-9]`).
- ✨ `\D` : Matches any non-digit character (equivalent to `[^0-9]`).

- ✨ `\w` : Matches any word character (alphanumeric and underscore).
- ✨ `\W` : Matches any non-word character (not alphanumeric or underscore).
- ✨ `\s` : Matches any whitespace character (space, tab, newline).
- ✨ `\S` : Matches any non-whitespace character.
- ✨ `^` : Matches the start of a string.
- ✨ `$` : Matches the end of a string.
- ✨ `|` : Acts as a logical OR operator in regex patterns.
- ✨ `{n}` : Matches exactly `n` occurrences of the preceding element.
- ✨ `{n,}` : Matches `n` or more occurrences of the preceding element.
- ✨ `{n,m}` : Matches between `n` and `m` occurrences of the preceding element.
- ✨ `(abc)` : Groups the characters `abc` together, allowing you to apply quantifiers or refer to the group later.

- ✨ `(?i)` : Makes the regex case-insensitive.
- ✨ `(?s)` : Allows the dot `.` to match newline characters, enabling multi-line matching.
- ✨ `(?m)` : Enables multi-line mode, allowing `^` and `$` to match the start and end of each line, respectively.
- ✨ `(?x)` : Allows whitespace and comments in the regex pattern for better readability.
- ✨ `(?<name>...)` : Creates a named capturing group, allowing you to refer to the group by name in the replacement string or when accessing matches.
- ✨ `(?#comment)` : Adds a comment in the regex pattern, which is ignored during matching.
- ✨ `(?=...)` : Positive lookahead assertion, checks if a pattern follows without including it in the match.
- ✨ `(?!...)` : Negative lookahead assertion, checks if a pattern does not follow without including it in the match.

More Regex: <https://regex101.com>

👉 Matching Strings with Regex

✨ The `matches()` method checks if the entire string matches a regex pattern.

✨ It returns `true` if the string matches the pattern, otherwise `false`.

Example: Matching a string against a regex pattern.

```
String s = "Hello123";  
boolean isMatch = s.matches("[A-Za-z]+\d+");  
System.out.println(isMatch); // Output: true
```

👉 Splitting Strings with Regex

✨ The `split()` method can be used to find substrings that match a regex pattern by splitting the string based on non-matching characters. It returns an array of substrings that match the pattern.

Example: Finding a substring that matches a regex pattern.

```
String s = "Hello123 World456";  
String[] numbers = s.split("\\D+");  
System.out.println(Arrays.toString(numbers)); // Output: [123, 456]
```

👉 Replace Strings with Regex

✨ The `replaceAll()` method replaces all occurrences of a regex pattern in a string with a specified replacement string.

✨ It returns a new string with the replacements made.

Example: Replacing all digits in a string with asterisks.

```
String s = "Hello123 World456";  
String replaced = s.replaceAll("\\d", "*");  
System.out.println(replaced); // Output: Hello*** World***
```

Output:

```
Hello*** World***
```

👉 Conversion between Strings and Arrays

✨ The `toCharArray()` method converts a string to a character array.

```
String s = "Hello";  
char[] chars = s.toCharArray();  
System.out.println(Arrays.toString(chars)); // Output: [H, e, l, l, o]
```

✨ `String.valueOf(char[] chars)` : Converts a character array to a string.

```
char[] chars = {'H', 'e', 'l', 'l', 'o'};  
String s = String.valueOf(chars);  
System.out.println(s); // Output: Hello
```

👉 Converting Characters and Numeric Values to Strings

✨ The `Character.toString()` method converts a character to a string.

```
char c = 'A';  
String charString = Character.toString(c);  
System.out.println(charString); // Output: A
```

✨ The `String.valueOf()` method can also be used to convert numeric values to strings.

```
int num = 42;  
String numString = String.valueOf(num);  
System.out.println(numString); // Output: 42  
double pi = 3.14;  
String piString = String.valueOf(pi);  
System.out.println(piString); // Output: 3.14
```

👉 Formatting Strings

- ✨ The `String.format()` method is used to create formatted strings using placeholders for variables.
- ✨ It allows you to specify the format of the output, such as decimal places, padding, and alignment.

```
String formatted = String.format("Hello %s, your score is %.2f", "Alice", 95.5);  
System.out.println(formatted); // Output: Hello Alice, your score is 95.50
```

- ✨ `printf(String format, Object... args)` : Prints a formatted string to the console.

```
System.out.printf("Hello %s, your score is %.2f%n", "Alice", 95.5);  
// Output: Hello Alice, your score is 95.50
```

10.11. The StringBuilder and StringBuffer Classes

👉 The **StringBuilder** and **StringBuffer** Classes

✨ **StringBuilder** and **StringBuffer** are mutable sequences of characters. They are used for creating and manipulating dynamic string content.

✨ **Key differences:**

✨ `StringBuilder` is faster and not synchronized, making it suitable for single-threaded applications.

✨ `StringBuffer` is synchronized, making it thread-safe and suitable for multi-threaded applications.

Performance Comparison

```
int n = 1_000_000;
long t1 = System.currentTimeMillis();
StringBuilder sb = new StringBuilder();
for (int i = 0; i < n; i++) sb.append("a");
System.out.println("StringBuilder: " + (System.currentTimeMillis() - t1) + "ms");

long t2 = System.currentTimeMillis();
StringBuffer sbf = new StringBuffer();
for (int i = 0; i < n; i++) sbf.append("a");
System.out.println("StringBuffer: " + (System.currentTimeMillis() - t2) + "ms");
```

Output:

```
StringBuilder: 9ms
StringBuffer: 22ms
```

👉 **Modifying Strings in the StringBuilder**

✨ **StringBuilder** allows for efficient string manipulation without creating new string objects. It provides methods to append, insert, delete, replace, and reverse characters in the sequence.

Example: Modifying a StringBuilder

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");
sb.insert(5, ",");
sb.delete(5, 6);
sb.replace(5, 11, " Java");
sb.reverse();
System.out.println(sb.toString()); // Output: avaJ olleH
```

👉 `toString()`, `capacity()`, `length()`, `setLength()`, and `charAt()`

✨ **StringBuilder** provides methods to convert the builder to a `String`, check its capacity, length, set the length, and access characters at specific indices.

Example: StringBuilder Methods

```
StringBuilder sb = new StringBuilder(50);  
System.out.println("Capacity: " + sb.capacity()); // Output: 50  
sb.append("Hello");  
System.out.println("Length: " + sb.length()); // Output: 5
```

TIP: Chain Method

✨ A class that provides methods to modify its state can allow method chaining, enabling multiple method calls in a single statement.

Example: Chaining Methods with StringBuilder

```
StringBuilder sb = new StringBuilder("Hello")
    .append(" World")
    .insert(5, ",")
    .replace(5, 11, " Java")
    .reverse();
System.out.println(sb.toString());
```

Output:

```
avaJ ,olleH
```

End of the Chapter